Validation of the Kinematics of a Wheeled Mobile Robot

Name: Parthib Kumar Dey
SR Number: 26787
Programme: M.Tech, Smart Manufacturing
Module: Wheeled mobile robot kinematics

September 24, 2026

1 Introduction

1.1 Aim

To validate the forward kinematics of a differential-drive mobile robot by estimating its pose (x, y, φ) and velocity (v_x, v_y) from wheel-encoder counts, while the robot executes a fixed manoeuvre: move forward 100 cm, turn right 90°, then move forward 50 cm. The estimated final pose is compared with the ideal pose $(x, y, \varphi) = (1.0\,\text{m}, -0.5\,\text{m}, -90°)$.

1.2 Overview of Approach

The robot is a two-wheeled differential-drive platform driven by an Arduino Mega. Each wheel carries a slotted-disc encoder read by an LM393 comparator module, giving 20 counts per wheel revolution. The count of each wheel is accumulated in an interrupt service routine (ISR). At a fixed minimum interval (30 ms) the main loop reads the count increments, converts them to wheel displacements and velocities, applies the differential-drive kinematic equations, and integrates the result to update the pose. The three motion stages are closed on the estimated odometry: the forward stages stop when the travelled distance reaches the target, and the turn stops when the integrated heading change reaches −90°. The pose and velocities are printed over the serial port every 10 cm.

2 Hardware Used

Component	**Model / Part**	**Qty**	**Purpose**
Microcontroller board	Arduino Mega	1	Odometry computation and control
Motor driver	Dual H-bridge (ENA/ENB, IN1–IN4)	1	Drives the two DC motors
DC geared motors	with drive wheels	2	Left and right drive
Wheels	∅ 65 mm	2	Ground contact
Encoder module	LM393 slot sensor, 20-slot disc	2	Wheel rotation counts

Table 1: Components used in the build.

Parameter	**Symbol**	**Value**
Encoder resolution	N	20 counts/rev
Wheel radius	r	0.0325 m
Wheel base	L	0.135 m
Forward PWM	–	150/255
Turn PWM	–	130/255
Update interval	$\Delta t_{\min}$	30 ms

Table 2: Robot and controller parameters.

3 Logic Used

3.1 Mathematical Model

Let ΔN_L and ΔN_R be the encoder count increments of the left and right wheels over a time step Δt. The robot pose is (x, y, φ), where φ is the heading measured from the x-axis (positive anticlockwise, so a right turn is negative).

Wheel rotation and displacement.

$$\Delta\alpha_L = \frac{2\pi\,\Delta N_L}{N}, \qquad \Delta\alpha_R = \frac{2\pi\,\Delta N_R}{N} \tag{1}$$

$$\Delta s_L = r\,\Delta\alpha_L, \qquad \Delta s_R = r\,\Delta\alpha_R \tag{2}$$

Wheel velocities.

$$\omega_L = \frac{2\pi\,\Delta N_L}{N\,\Delta t}, \qquad \omega_R = \frac{2\pi\,\Delta N_R}{N\,\Delta t}, \qquad v_L = r\,\omega_L, \qquad v_R = r\,\omega_R \tag{3}$$

Robot linear and angular velocity (differential-drive forward kinematics).

$$v = \frac{v_R + v_L}{2}, \qquad \omega = \frac{v_R - v_L}{L} \tag{4}$$

World-frame velocity. The heading at the midpoint of the interval is used to reduce the integration error, especially during turns:

$$\varphi_{\text{mid}} = \varphi_k + \frac{\omega\,\Delta t}{2}, \qquad v_x = v\cos\varphi_{\text{mid}}, \qquad v_y = v\sin\varphi_{\text{mid}} \tag{5}$$

Pose update (numerical integration).

$$x_{k+1} = x_k + v_x\,\Delta t, \qquad y_{k+1} = y_k + v_y\,\Delta t, \qquad \varphi_{k+1} = \varphi_k + \omega\,\Delta t \tag{6}$$

Stopping conditions.

$$d = \sqrt{(x - x_0)^2 + (y - y_0)^2} \geq d_{\text{target}} \quad \text{(forward stages)}, \qquad \varphi - \varphi_0 \leq -\frac{\pi}{2} \quad \text{(right turn)} \tag{7}$$

Resolution and expected values. Distance per encoder count:

$$\Delta s_{\text{count}} = \frac{2\pi r}{N} = \frac{2\pi(0.0325)}{20} \approx 0.0102\,\text{m} \tag{8}$$

A one-count difference between the wheels changes the heading by $\Delta s_{\text{count}}/L \approx 0.0756\,\text{rad} \approx 4.3^\circ$. For an in-place turn (left wheel forward, right wheel backward) the heading change is $\Delta\varphi = (\Delta s_R - \Delta s_L)/L$. A 90° right turn therefore needs $\Delta s_L - \Delta s_R = L\pi/2 \approx 0.212\,\text{m}$, i.e. about 21 counts of combined wheel travel (≈ 10 counts per wheel).

The ideal manoeuvre, starting at $(0, 0, 0^\circ)$, gives

$$(0, 0, 0^\circ) \xrightarrow{1.0\,\text{m}} (1.0,\ 0,\ 0^\circ) \xrightarrow{-90^\circ} (1.0,\ 0,\ -90^\circ) \xrightarrow{0.5\,\text{m}} (1.0,\ -0.5,\ -90^\circ). \tag{9}$$

3.2 Implementation Notes

- **Direction sensing:** the encoder is single-channel, so the sign of each count is taken from the commanded motor direction (+1 forward, −1 backward), set inside the motor functions and read by the ISR.
- **Debounce:** an edge arriving within $1500\,\mu$s of the previous accepted edge is ignored, to reject comparator chatter.
- **Update gating:** kinematics run only when at least 30 ms have elapsed, so more counts accumulate per update and the ± 1 count quantisation error on ω is reduced.
- **Safety timeouts:** the forward stage (15 s) and turn stage (10 s) stop the motors if the target is not reached.

3.3 Algorithmic Structure / Pseudocode

Algorithm 1: Main experiment

Input: Parameters N, r, L; target distances $d_1 = 1.0\,\text{m}$, $d_2 = 0.5\,\text{m}$
Output: Pose (x, y, φ) and velocities (v_x, v_y) printed every 10 cm

Attach encoder ISRs on both wheels (rising edge);
$x, y, \varphi \leftarrow 0$; $N_L^{prev}, N_R^{prev} \leftarrow$ current counts; $t^{prev} \leftarrow$ now;
MoveForward(d_1);
TurnRight90();
MoveForward(d_2);
Print final state;

Algorithm 2: Encoder ISR (left; right is identical)

$t \leftarrow$ micros();
if $t - t_L^{last} < 1500\,\mu s$ **then return**;
$t_L^{last} \leftarrow t$;
if $dir_L > 0$ **then** $N_L \leftarrow N_L + 1$;
else if $dir_L < 0$ **then** $N_L \leftarrow N_L - 1$;

Algorithm 3: UpdateOdometry

$t \leftarrow$ millis();
if $t - t^{prev} < 30\,ms$ **then return**;
$\Delta t \leftarrow (t - t^{prev})/1000$; $t^{prev} \leftarrow t$;
Read N_L, N_R with interrupts disabled;
$\Delta N_L \leftarrow N_L - N_L^{prev}$; $\Delta N_R \leftarrow N_R - N_R^{prev}$; update N^{prev};
$\Delta s_{L,R} \leftarrow r \cdot 2\pi\, \Delta N_{L,R}/N$;
$v_{L,R} \leftarrow \Delta s_{L,R}/\Delta t$;
$v \leftarrow (v_R + v_L)/2$; $\omega \leftarrow (v_R - v_L)/L$;
$\varphi_{mid} \leftarrow \varphi + \omega\,\Delta t/2$;
$v_x \leftarrow v\cos\varphi_{mid}$; $v_y \leftarrow v\sin\varphi_{mid}$;
$x \leftarrow x + v_x\Delta t$; $y \leftarrow y + v_y\Delta t$; $\varphi \leftarrow \varphi + \omega\,\Delta t$;

Algorithm 4: MoveForward(d_{target})

$(x_0, y_0) \leftarrow (x, y)$; reset odometry timing; drive both wheels forward ($dir = +1$);
repeat
  UpdateOdometry();
  $d \leftarrow \sqrt{(x - x_0)^2 + (y - y_0)^2}$;
  if *d has advanced by another 10 cm* **then**
    print x, y, φ, v_x, v_y
  end
until $d \geq d_{target}$ ***or*** *timeout*;
Stop motors; print state (and a warning if timed out);

Algorithm 5: TurnRight90

$\varphi_0 \leftarrow \varphi$; reset odometry timing;
Left wheel forward ($dir_L = +1$), right wheel backward ($dir_R = -1$);
repeat
  UpdateOdometry();
until $\varphi - \varphi_0 \leq -\pi/2$ ***or*** *timeout*;
Stop motors; print state (and a warning if timed out);

4 Code

I have uploaded the code in the folder.

5 Video

1. Video 1 – Arduino serial monitor output (X, Y, φ, v_x, v_y every 10 cm)
2. Video 2 – Robot with wheels off the ground, held on a support
3. Video 3 – Robot running the manoeuvre on the ground